

Bitácora de Diseño Cerradura Electrónica

Jose Andrés Chavarría Gamboa

Isaac Andrés Arias Picado

Escuela de Ingeniería en Computadores, Instituto Tecnológico de Costa Rica (TEC)

CE-1107 Fundamentos de Arquitectura de Computadores — Profesor: Luis Alberto Chavarría Zamora

Repositorio: https://github.com/JoseChava/Bit-cora_Proyecto1_Fundamentos

I. 2 DE SEPTIEMBRE

Objetivo: el día de hoy se busca iniciar con la planeación del diseño general del sistema de cerradura con clave. Se toman decisiones de diseño sobre el modelo y se analiza el funcionamiento de cada bloque por separado.

Se repasó el enunciado del proyecto y se definió cómo se iba a resolver cada bloque del diagrama general (sensores → circuito combinacional → BCD → display de 7 segmentos, más la rama de desacople/accionador). Las decisiones principales:

- Por ahora el sistema es completamente combinacional, sin ningún elemento de memoria: los 3 dígitos de la contraseña se fijan directo con dos bancos de DIP-switch, y esa combinación entra directo a las compuertas del comparador, que a su vez alimentan al decodificador de 7 segmentos.
- Se fija como contraseña de prueba 512 para cerrar y 450 para abrir.
- Se trabaja únicamente con compuertas CMOS (serie 74HC) en todo el circuito, sin mezclar con TTL: la CMOS tiene una impedancia de entrada muchísimo más alta, un rango de voltaje de alimentación distinto y márgenes de ruido distintos a los de TTL, así que una salida de una familia no necesariamente entrega el nivel que la otra reconoce como válido. Además, toda entrada CMOS que no se vaya a usar se debe poner en GND, esto para evitar fallos en el dispositivo por el ruido generado al dejar la entrada al aire. Si en alguna etapa hiciera falta interconectar con otra familia, se documentará el uso de un traductor de niveles.
- Cada etapa se diseña primero como tabla de verdad, se reduce con mapas K y/o lógica booleana y se simula antes de armar en protoboard.
- El diseño del sistema, las decisiones de arquitectura y todas las tablas de verdad son trabajo propio de los autores; se hizo uso de IA únicamente para verificar/simplificar expresiones booleanas ya obtenidas y para apoyar la redacción de esta bitácora.

II. 9 DE SEPTIEMBRE — VERIFICADOR DE CONTRASEÑA

Objetivo: armar el comparador que decide si la combinación puesta en los switches corresponde al código de abrir, al de cerrar, o a ninguno de los dos.

Los dos DIP-switch de 5 posiciones (de las cuales solo 9 serán utilizadas) ponen directo los 3 bits de cada uno de los 3 dígitos de la contraseña sobre las compuertas, sin ningún registro ni elemento de memoria de por medio.

El primer modelo que se probó (Fig. 1) no quemaba el código directo en compuertas: cada bit del switch se comparaba mediante una XNOR contra un bit de referencia aparte (*contra_a* para el código de abrir, *contra_c* para el de cerrar), y solo si todas las comparaciones coincidían se activaba *abierto* o *cerrado*. Esto en teoría permitía mover el código con solo recablear esas referencias, pero como el proyecto nunca buscó que la contraseña fuera configurable, se decidió simplificar: en el modelo final (Fig. 2) se elimina por completo el banco de referencias y el código queda “quemado” directo con compuertas NOT fijas en cada entrada, según el patrón de cada código.

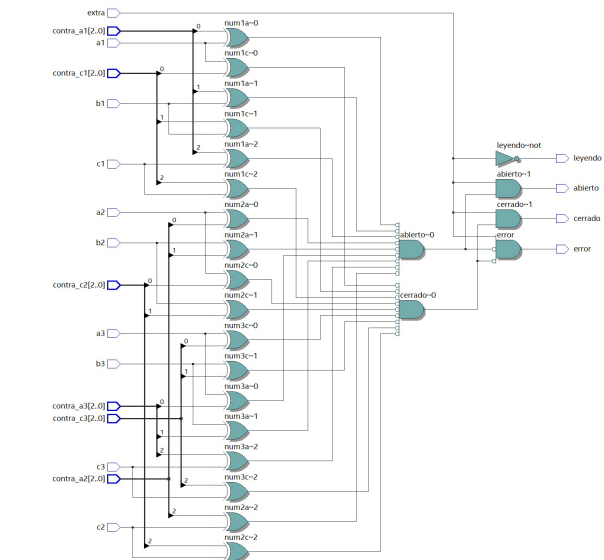


Fig. 1. Primer modelo del verificador: comparación bit a bit contra referencias configurables (*contra_a*, *contra_c*) mediante XNOR.

Llamando *A, B, C* a los 3 bits del primer dígito, *D, E, F* a los del segundo y *G, H, I* a los del tercero, en el modelo

final el código queda “quemado” con compuertas NOT en las entradas correspondientes (Fig. 2), y la comparación contra cada código fijo es un solo AND de 9 literales (directos o negados según el bit valga 1 o 0 en el código):

$$\begin{aligned} matchC &= A \overline{B} C \overline{D} \overline{E} F \overline{G} H \overline{I} \quad (\text{patrón de 512}) \\ matchA &= A \overline{B} \overline{C} D \overline{E} F \overline{G} \overline{H} \overline{I} \quad (\text{patrón de 450}) \end{aligned} \quad (1)$$

donde en cada caso los literales que van negados dependen del bit correspondiente de cada código (por eso $matchA$ define qué combinación abre, y $matchC$ qué combinación cierra: queda confirmado en el propio cableado del comparador). De ahí salen directo las tres salidas, sin ningún habilitador extra de por medio:

$$\begin{aligned} cerrado &= matchC \\ abierto &= matchA \\ error &= \overline{matchC + matchA} \end{aligned} \quad (2)$$

($error$ se prende cuando la combinación de switches no calza con ninguno de los dos códigos).

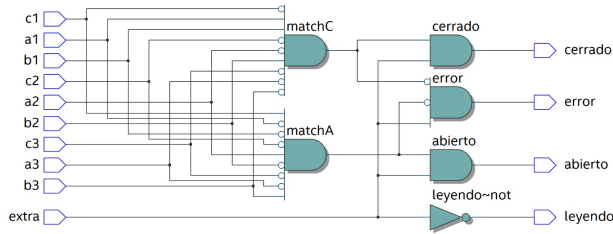


Fig. 2. Comparador de código fijo (matchC / matchA) y las salidas cerrado, error, abierto.

Como la función completa depende de 9 bits, no tiene caso mostrar la tabla de verdad completa (512 filas). Alcanza con verla dígito por dígito: cada bloque de 3 bits solo debe encender en el valor exacto de ese dígito. La Tabla I muestra esa comparación para el primer dígito (A, B, C) con los dos patrones usados en el código real (5 para cerrar, 4 para abrir).

TABLE I
COINCIDENCIA DEL PRIMER DÍGITO CONTRA LOS PATRONES DE CADA CÓDIGO

A	B	C	cierra (5)	abre (4)
0	0	0	0	0
0	0	1	0	0
0	1	0	0	0
0	1	1	0	0
1	0	0	0	1
1	0	1	1	0
1	1	0	0	0
1	1	1	0	0

De las 8 combinaciones posibles solo una enciende cada salida, así que el mapa K (Tabla II) queda con un solo 1 aislado, sin nada que agrupar: la expresión ya es mínima

tal como quedó cableada, $A \overline{B} C$ para el patrón de cerrar. Análogamente, para el patrón de abrir el único 1 cae en $A = 1$, $BC = 00$, dando $A \overline{B} \overline{C}$.

TABLE II
MAPA K DE LA COINCIDENCIA DEL PRIMER DÍGITO (PATRÓN 5, CERRAR)

A\BC	00	01	11	10
0	0	0	0	0
1	0	1	0	0

El mismo patrón se repite para el segundo dígito (D, E, F , códigos 1 para cerrar y 5 para abrir, Tabla III) y para el tercero (G, H, I , códigos 2 para cerrar y 0 para abrir, Tabla V): en ambos casos, de las 8 combinaciones posibles solo una enciende cada salida.

TABLE III
COINCIDENCIA DEL SEGUNDO DÍGITO CONTRA LOS PATRONES DE CADA CÓDIGO

D	E	F	cierra (1)	abre (5)
0	0	0	0	0
0	0	1	1	0
0	1	0	0	0
0	1	1	0	0
1	0	0	0	0
1	0	1	0	1
1	1	0	0	0
1	1	1	0	0

TABLE IV
MAPA K DE LA COINCIDENCIA DEL SEGUNDO DÍGITO (PATRÓN 1, CERRAR)

D\EF	00	01	11	10
0	0	1	0	0
1	0	0	0	0

Otra vez el mapa K de la Tabla IV queda con un único 1 en $D = 0$, $EF = 01$, dando $\overline{D} \overline{E} F$; para el patrón de abrir el único 1 cae en $D = 1$, $EF = 01$, dando $D \overline{E} F$.

TABLE V
COINCIDENCIA DEL TERCER DÍGITO CONTRA LOS PATRONES DE CADA CÓDIGO

G	H	I	cierra (2)	abre (0)
0	0	0	0	1
0	0	1	0	0
0	1	0	1	0
0	1	1	0	0
1	0	0	0	0
1	0	1	0	0
1	1	0	0	0
1	1	1	0	0

TABLE VI
MAPA K DE LA COINCIDENCIA DEL TERCER DÍGITO (PATRÓN 2, CERRAR)

$G \backslash HI$	00	01	11	10
0	0	0	0	1
1	0	0	0	0

El mapa K de la Tabla VI vuelve a quedar con un único 1, en $G = 0$, $HI = 10$, dando $\overline{G} H \overline{I}$; para el patrón de abrir el único 1 cae en $G = 0$, $HI = 00$, dando $\overline{G} \overline{H} \overline{I}$.

Como los tres bloques de 3 bits (A, B, C ; D, E, F ; G, H, I) usan variables completamente independientes entre sí, y cada uno ya es una función de un solo minitérmino — es decir, ya mínima, sin ningún otro 1 adyacente con el cual agruparse en su propio mapa K —, el producto AND de los tres términos tampoco se puede reducir más: ninguna ley de álgebra booleana (absorción, consenso, idempotencia) permite simplificar un AND de literales que pertenecen a variables distintas. Así, *matchC* y *matchA* (Ecuación 1) ya son la expresión mínima de todo el comparador, sin necesidad de construir (ni de que exista) un mapa K de 9 variables.

Se simuló antes de armar nada físico, y se confirmó que abierto y cerrado nunca se prenden al mismo tiempo (por construcción, *matchA* y *matchC* no pueden dar 1 a la vez). Este es el circuito que se terminó implementando físicamente.

III. 15 DE SEPTIEMBRE — DECODIFICADOR DE 7 SEGMENTOS

Objetivo: traducir las tres salidas del comparador (abierto/cerrado/error) a los segmentos $a-g$ del display, mostrando A (abriendo), C (cerrando) o E (clave inválida).

Se hizo la tabla de verdad con 3 entradas: $A=abierto$, $B=cerrado$ y $C=error$. Por cómo quedó armado el comparador, exactamente una de las tres siempre está en 1 (nunca las tres en 0, ni dos a la vez), así que las otras 5 combinaciones de (A, B, C) no pueden pasar y se dejan como no importa ($\times$), ver Tabla VII.

TABLE VII
TABLA DE VERDAD DEL DECODIFICADOR DE 7 SEGMENTOS

A	B	C	a	b	c	d	e	f	g
0	0	0	$\times$	$\times$	$\times$	$\times$	$\times$	$\times$	$\times$
1	0	0	1	1	1	0	1	1	1
0	1	0	1	0	0	1	1	1	0
0	0	1	1	0	0	1	1	1	1
1	1	0	$\times$	$\times$	$\times$	$\times$	$\times$	$\times$	$\times$
1	0	1	$\times$	$\times$	$\times$	$\times$	$\times$	$\times$	$\times$
0	1	1	$\times$	$\times$	$\times$	$\times$	$\times$	$\times$	$\times$
1	1	1	$\times$	$\times$	$\times$	$\times$	$\times$	$\times$	$\times$

Un primer borrador (sin reducir, Fig. 3) simplemente sacó seg_d y seg_g como un OR directo de $A/B/C$. Por ejemplo, tomando solo los minitérminos donde el segmento debe en-

cender (sin usar todavía los $\times$), esos dos quedaban así antes de reducir:

$$seg_d \text{ (antes)} = \overline{A} B \overline{C} + \overline{A} \overline{B} C \quad (3)$$

$$seg_g \text{ (antes)} = A \overline{B} \overline{C} + \overline{A} \overline{B} C \quad (4)$$

Al meter los $\times$ en el mapa K esas expresiones se reducen mucho:

$$\begin{aligned} seg_a &= 1 & seg_e &= 1 \\ seg_b &= A & seg_f &= 1 \\ seg_c &= A & seg_g &= \overline{B} \\ seg_d &= \overline{A} & & \end{aligned} \quad (5)$$

Es decir, a , e y f quedan siempre encendidos (no hace falta ni una compuerta para ellos), y b , c , d , g terminan dependiendo de una sola variable cada uno, gracias a que las combinaciones imposibles se pudieron usar a favor en el mapa K. Esta reducción también se corrió por un modelo de IA junto con el mapa K para confirmar que no quedara ningún término de más, y coincidió.

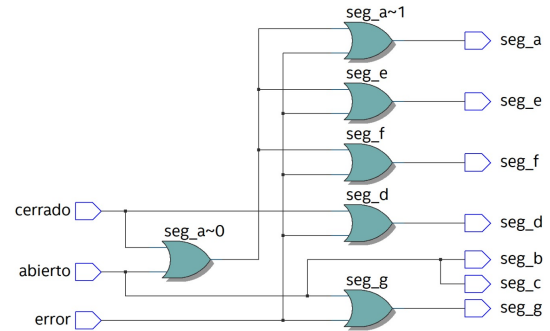


Fig. 3. Borrador inicial sin reducir del decodificador de 7 segmentos.

IV. 16 DE SEPTIEMBRE — INTEGRACIÓN Y PRUEBAS

Objetivo: armar en protoboard el verificador de contraseña junto con el decodificador de 7 segmentos ya reducido, y probar que funcione con los dos códigos.

Se montó todo en dos protoboards acopladas (Fig. 4): la de la izquierda tiene los dos DIP-switch con el código puesto y el circuito integrado del comparador; la de la derecha tiene el decodificador de 7 segmentos y el display de salida. Cada banco de switches se conecta al positivo a través de una resistencia de pull-up de 10 k Ω , de forma que la entrada quede en alto cuando el switch está abierto y se vaya a bajo solo cuando se cierra; son las dos resistencias que se ven en la esquina superior izquierda de la protoboard.

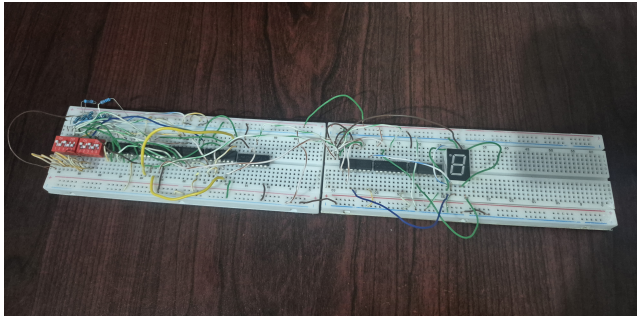


Fig. 4. Verificador de contraseña (izquierda) y decodificador de 7 segmentos (derecha) ya integrados en protoboard.

Se probó con el código 450 (abre, muestra “A”), con 512 (cierra, muestra “C”) y con una combinación que no coincide con ninguno (muestra “E”). En la primera prueba, antes de terminar de cablear las tres salidas del comparador hacia el decodificador, el display quedó mostrando todos los segmentos prendidos (como un “8”), lo que sirvió para confirmar que el display en sí estaba bien y que el problema era ese cableado, no los circuitos integrados. Ya corregido, los tres casos funcionaron bien.

V. 24 DE SEPTIEMBRE — MÓDULO BASE 6 A BINARIO Y CADENA DE DESPLAZAMIENTO (165/595)

Objetivo: reemplazar la entrada por DIP-switch por un teclado de 5 fotoresistencias (una por dedo), convertir cada toque a su valor binario de 3 bits, y agregar una etapa de almacenamiento secuencial con el 74HC165 y dos 74HC595 en cascada para armar la contraseña de 9 bits antes de compararla.

Hasta la sesión anterior la contraseña se ponía completa y en paralelo con dos DIP-switch. En esta sesión se cambió el modelo de entrada para que el usuario *toque un dedo a la vez* sobre 5 fotoresistencias: cada fotoresistencia representa un dígito del 1 al 5, y el dígito 0 queda implícito cuando no se toca ninguna. Cada dígito se codifica en 3 bits (base 6), y la contraseña final son 3 dígitos concatenados (9 bits en total).

A. Conversión base 6 a binario

La tabla de conversión de cada dedo a su valor de 3 bits es la siguiente:

TABLE VIII
CODIFICACIÓN DE CADA FOTORESISTENCIA (DÍGITO) A 3 BITS

Dígito	b_2	b_1	b_0
0 (ninguno)	0	0	0
1 (SW0)	0	0	1
2 (SW1)	0	1	0
3 (SW2)	0	1	1
4 (SW3)	1	0	0
5 (SW4)	1	0	1

De la tabla se ven directo las tres funciones booleanas (cada b_i es un OR de los switches donde ese bit vale 1):

$$\begin{aligned} b_0 &= SW_0 + SW_2 + SW_4 \quad (\text{dígitos } 1, 3, 5) \\ b_1 &= SW_1 + SW_2 \quad (\text{dígitos } 2, 3) \\ b_2 &= SW_3 + SW_4 \quad (\text{dígitos } 4, 5) \end{aligned} \quad (6)$$

Estas tres salidas entran directo, en paralelo, a las entradas del 74HC165.

B. Cadena de desplazamiento: 74HC165 + 2×74HC595

Los 3 bits del dígito actual entran al 74HC165 en paralelo. Con un switch manual se selecciona el modo: *load* para capturar el dígito, y *shift* para sacarlo serialmente. La salida serial del 165 entra al primer 74HC595, y la salida serial de éste entra al segundo; los dos comparten el mismo pulso de latch (STCP). Así, los 9 bits de la contraseña quedan disponibles en paralelo a la salida de los dos 595.

El flujo por cada dígito es:

- 1) Colocar el dedo en la fotoresistencia correspondiente (modo *load*).
- 2) Pasar a modo *shift* y dar 3 pulsos de clock: el dígito se desplaza hacia los 595.
- 3) Repetir para los 3 dígitos de la contraseña.
- 4) Al final, pulsar STCP para que los 9 bits aparezcan en paralelo en la salida de los 595, listos para entrar al comparador.

Implementación de los clocks

Los clocks de cada dispositivo se implementaron **por aparte, con botones independientes**: un botón para el SHCP del 74HC165 y un botón para el SHCP de cada uno de los dos 74HC595. Se descartó la idea de compartir un único clock entre todos porque se presentaron problemas de **retraso** y de **flancos de subida**: al pulsar un solo botón, la señal llegaba a los distintos chips con retardos ligeramente distintos por el cableado y las capacitancias parásitas del protoboard, y en algunos casos el flanco de subida se deformaba lo suficiente como para que un chip registrara el pulso y otro no en el mismo ciclo. Al separar los clocks con botones independientes se pudo controlar manualmente el orden exacto de los pulsos, garantizando que el 165 cargara o shiftara antes de que los 595 registraran el bit correspondiente. El pulso de latch (STCP) sí se mantuvo compartido entre los dos 595, porque ahí lo que importa es que ambos actualicen su salida al mismo tiempo, no cuándo entra cada bit.

Verificación en Quartus

Se escribió el módulo completo en Verilog y se sintetizó en Quartus Prime para revisar el diagrama RTL. La jerarquía quedó: Prueba → (deco_switches, control, hc165, hc595_x2), con las compuertas OR del decodificador de switches visibles directamente en el RTL para confirmar la reducción de las ecuaciones de b_0 , b_1 y b_2 . La salida del 165 es un solo bit serial, y la salida del bloque 595 es un bus de 9 bits, tal como se esperaba del análisis teórico. El único detalle de sintaxis que hubo que corregir fue el rango del vector de

switches (se declaró [4:0] porque son 5 fotoresistencias, no 6).

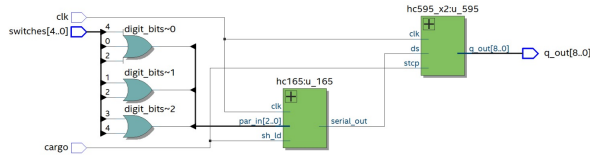


Fig. 5. Diagrama RTL del módulo base 6 a binario y la cadena 165/595, generado en Quartus Prime.

VI. 25 DE SEPTIEMBRE — DISEÑO DEL DESACOPLE Y DEL PUENTE H

Objetivo: definir cómo se desacopla la lógica del motor y cómo se invierte su giro para abrir y cerrar la puerta.

El enunciado pide que el desacople se haga con componentes discretos, no únicamente mecánicos o magnéticos, así que se descartó usar un relé como desacople. Se eligió un optoacoplador PC817: por dentro tiene un LED infrarrojo y un fototransistor, y la única conexión entre ambos lados es la luz, por lo que la tierra de la lógica queda aislada de la tierra del motor. Para invertir el giro se usa un puente H con transistores bipolares: dos PNP BD140 arriba (emisor a +6 V) y dos NPN BD139 abajo (emisor a GND). Las decisiones de esta etapa fueron:

- Dos optoacopladores, uno para *abierto* y otro para *cerrado*. Cada uno activa una diagonal del puente (un PNP y el NPN del lado contrario).
- Resistencia de 330 Ω en serie con el LED del optoacoplador (≈ 10 mA), 470 Ω en cada base y 10 k Ω entre base y emisor de cada transistor, para que se apague bien cuando su optoacoplador no conduce.
- Diodos 1N4007 de rueda libre entre colector y emisor de cada transistor, con el cátodo hacia el lado más positivo, para absorber el pico inductivo del motor al apagarlo.
- El motor se alimenta aparte, con una batería de 9 V y un regulador 7806 que entrega 6 V, con capacitores de 0,33 μ F en la entrada, 0,1 μ F en la salida, 470 μ F entre los rieles y 100 nF entre los bornes del motor para filtrar el ruido de las escobillas.
- Como las salidas del comparador nunca están en 1 al mismo tiempo, nunca se activan las dos diagonales a la vez, lo que evita un cortocircuito en el puente.

VII. 27 DE SEPTIEMBRE — SIMULACIÓN EN LTSPICE

Objetivo: simular el desacople y el puente H para confirmar que el motor gira en ambos sentidos según la señal que llegue del comparador.

Se armó el esquemático en LTspice (Fig. 6) usando modelos de los fabricantes para los transistores BD139/BD140, los diodos 1N4007 y los optoacopladores PC817. Como LTspice no trae un motor DC, se representó con una resistencia y una bobina en serie. Las señales de abrir y cerrar se simularon con dos fuentes de 5 V que se encienden una a la vez, y se dejó

la tierra de la lógica separada de la del motor, igual que en el circuito real.

Mientras se armaba el esquemático aparecieron algunos errores de conexión: dos diodos invertidos y los optoacopladores conectados a transistores equivocados. Una vez corregidos, la simulación mostró lo esperado (Tabla IX): al activar la señal de abrir el motor recibe voltaje en un sentido, al activar la de cerrar lo recibe en el sentido contrario, y cuando ninguna está activa el motor queda apagado.

TABLE IX
RESULTADOS DE LA SIMULACIÓN EN LTSPICE

Estado	V_{motor} (V)	I_{motor} (mA)
Abrir	+4,64	+309
Cerrar	-4,64	-309
Reposo	0	0

Con esto se confirmó que el puente invierte correctamente el giro del motor, que nunca se activan las dos direcciones al mismo tiempo, y que la lógica y el motor quedan aislados gracias a los optoacopladores.

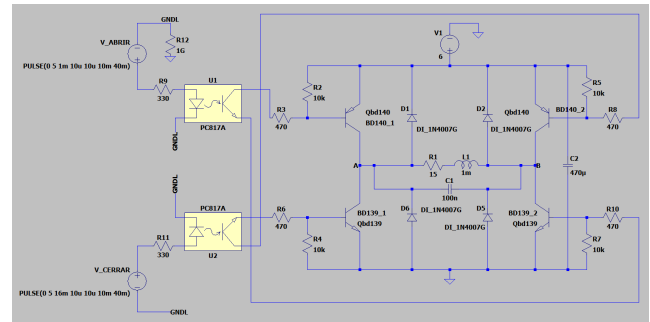


Fig. 6. Esquemático en LTspice del desacople con optoacopladores PC817 y del puente H.

VIII. 28 DE SEPTIEMBRE — ARMADO Y PRUEBAS DEL PUENTE H

Objetivo: armar el desacople y el puente H y verificar que cada parte funcione por separado antes de conectar el motor.

Como no se tenía multímetro, todas las pruebas se hicieron con un LED con una resistencia de 1 k Ω en serie, que enciende solo cuando pasa corriente. Se probó de lo más pequeño a lo más grande: primero cada transistor por separado, luego cada diagonal del puente, después los optoacopladores y al final el motor.

Identificación de pines

Al conectar los transistores, se hizo una mala identificación de los pines. Para confirmar la posición correcta de los pines, cada transistor se probó por aparte hasta confirmar cuál pata era la base, cuál el colector y cuál el emisor. Así también se verificó que los cuatro funcionaban.

Error de diseño encontrado

En el primer armado, las resistencias de base de dos transistores del mismo lado del puente quedaron conectadas a uno de los bornes del motor. Sin ninguna señal activa, el LED de prueba encendía débil: la corriente se colaba de un transistor al otro y los dos quedaban medio encendidos, lo que es un cortocircuito parcial. Se corrigió cambiando la forma de conectar el puente: ahora cada optoacoplador une directamente la base de un transistor de arriba con la del transistor de abajo del lado contrario, y ninguna resistencia va a los bornes del motor. Así, si el optoacoplador está apagado, ningún transistor enciende.

Otros problemas

- Uno de los transistores NPN se había conectado como si fuera PNP.
- Una resistencia de base había quedado sin conectar.
- Un optoacoplador parecía dañado, pero el problema era que sus patas no hacían buen contacto con el protoboard. Se movió a otra zona y funcionó.
- Uno de los LED de prueba estaba quemado, lo que explicaba algunos resultados raros.

En la Fig. 7 se muestra el puente armado durante estas pruebas.

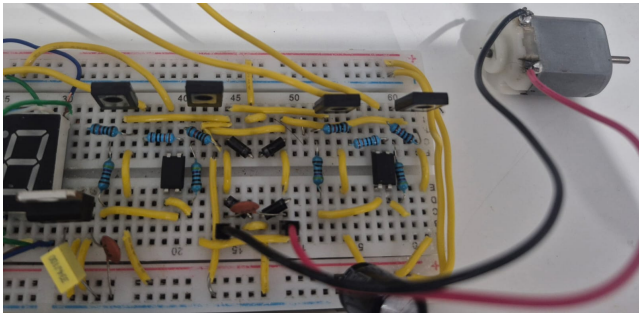


Fig. 7. Puente H y optoacopladores armados en protoboard durante las pruebas.

TABLE X
PRUEBAS DEL DESACOPLE Y DEL PUENTE H

Prueba	Resultado
Cada transistor por separado	Funciona
Sin señal activa	Todo apagado
Dirección abrir	Funciona
Dirección cerrar	Funciona
Optoacopladores en el puente	Funcionan
Motor con el puente	Gira en ambos sentidos

IX. 29 DE SEPTIEMBRE — INTEGRACIÓN Y PRUEBAS
FINALES

Objetivo: conectar las salidas del comparador al desacople y probar el sistema completo, desde la entrada de la contraseña hasta el giro del motor.

Las salidas del comparador ya estaban en one-hot (100 cerrado, 010 error, 001 abierto), así que no hizo falta agregar compuertas: la salida *abierto* se conectó al optoacoplador que activa la dirección de abrir, y la salida *cerrado* al que activa la de cerrar. La salida *error* solo va al display. Como nunca hay dos salidas en 1 al mismo tiempo, el motor nunca recibe las dos órdenes juntas. La lógica y el motor siguieron con fuentes separadas.

Se realizaron múltiples pruebas de funcionamiento utilizando un LED para revisar la salida del 165 en cada estado, y en todas el sistema respondió correctamente: el display mostró “A”, “C” o “E” según el código ingresado, y el motor giró en la dirección correcta al abrir y al cerrar. En la Fig. 8 se muestra el circuito completo terminado, en este caso en el estado “E”.

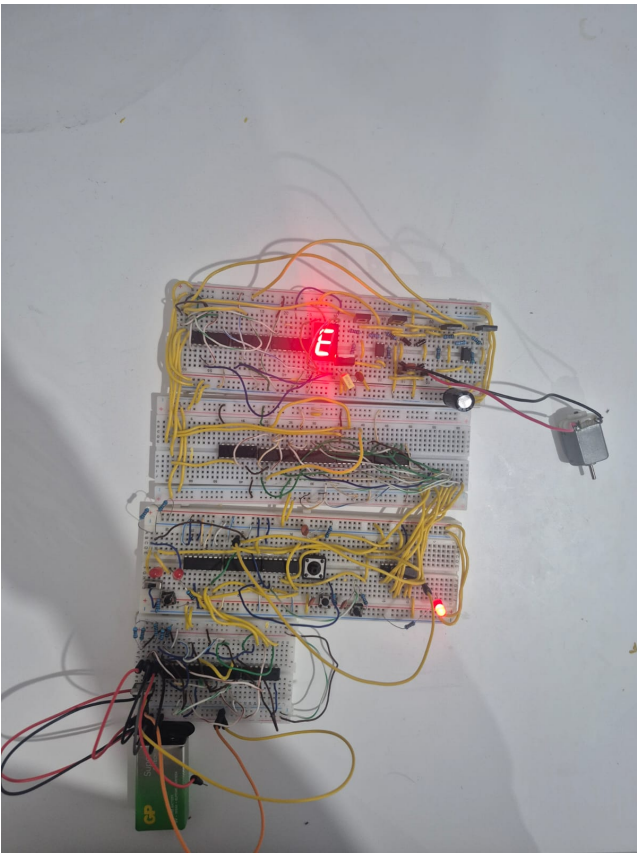


Fig. 8. Circuito completo en protoboard, mostrando el estado “E” (código inválido).